

App Dev Project Report

1. Student Details

Name: Pammi Tiwari

Roll Number: 22f2001271

Email: 22f2001271@ds.study.iitm.ac.in

About Me: I am a BS in Data Science and Application student at IIT Madras, learning the fundamentals of programming, analytics, and data-driven problem solving. I enjoy working on practical projects that help me apply my skills and explore new concepts in technology. I am always looking for opportunities to learn and improve in the field of data science.

2. Project Details

Problem Statement:

Hospitals often face confusion when managing patient details, doctor information, and appointment timings manually. This can cause mistakes, double bookings, and problems in keeping track of treatment records. This project aims to make these tasks easier through a simple web-based system.

Approach:

This system is built using Flask, Jinja2 templates, Bootstrap, and SQLite. It includes three main roles:

1. **Admin:** manages doctors, patients, and appointments
2. **Doctor:** views their schedule and updates patient treatment details
3. **Patient:** registers, searches for doctors or departments, and books appointments

The project is organized into separate routes, models, and templates. It also includes checks to stop double booking and keeps a full treatment history for each patient.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist in understanding certain SQLAlchemy queries, simplifying a few route functions, and improving the clarity of some parts of the code and documentation. The extent of AI/LLM usage is around 8–10%, mainly for guidance, explanations, and small code suggestions. All main development work and project structuring were completed independently.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework used to create routes, handle requests, and manage application flow.
SQLAlchemy	ORM used to define database models and interact with the SQLite database.
Jinja2	Template engine used for rendering dynamic HTML pages with data.
Bootstrap	Frontend framework used for styling, layout, forms, buttons, and responsive design.
SQLite	Lightweight local database used via SQLAlchemy for storing all project data.
Python datetime module	Used to manage dates, times, time ranges, and appointment scheduling logic.

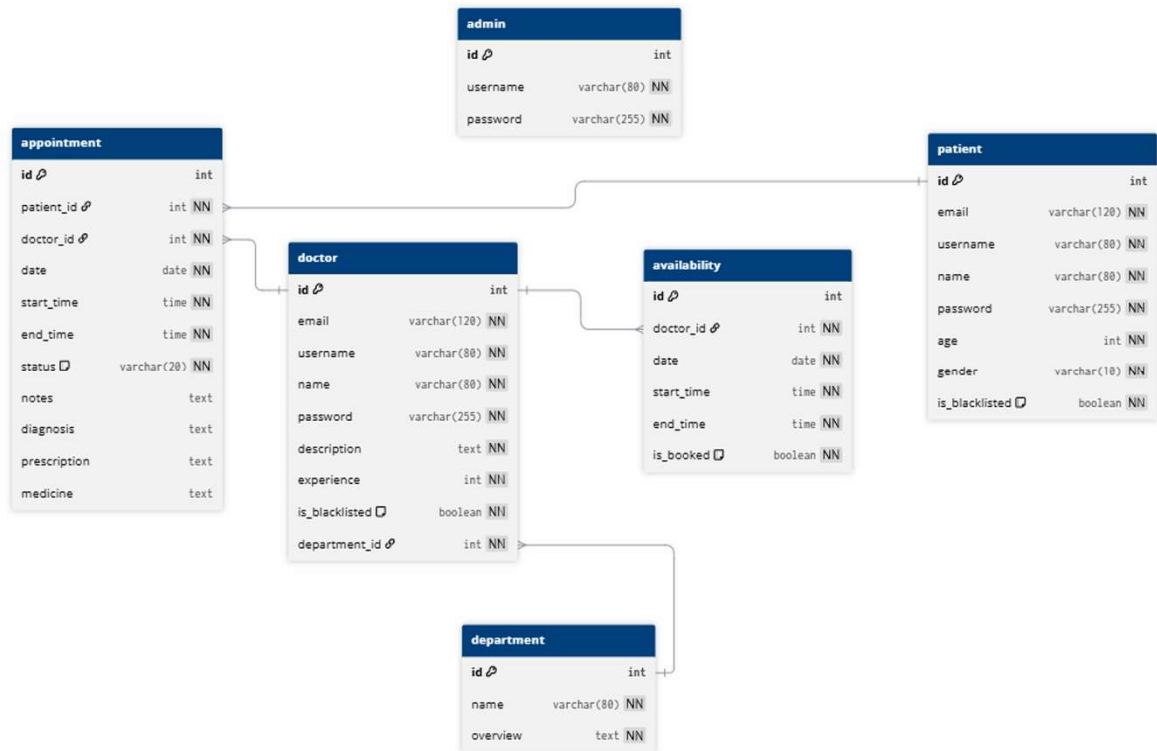
5. Database Schema / ER Diagram

Tables:

1. **Admin:** Stores admin login details — id, username, password.
2. **Patient:** Stores patient information — id, email, username, name, password, age, gender, is_blacklisted; relationship: appointments.
3. **Department:** Stores department information — id, name, overview; relationship: doctors.
4. **Doctor:** Stores doctor details — id, email, username, name, password, description, experience, is_blacklisted, department_id; relationships: appointments, availabilities.
5. **Availability:** Stores doctor availability slots — id, doctor_id, date, start_time, end_time, is_booked, **unique constraint:** (doctor_id, date, start_time).
6. **Appointment:** Stores appointments and treatment details — id, patient_id, doctor_id, date, start_time, end_time, status, notes, diagnosis, prescription, medicine..

Relationships:

- One-to-Many → Department → Doctor
- One-to-Many → Doctor → Availability
- One-to-Many → Doctor → Appointment
- One-to-Many → Patient → Appointment



Routes

1. Authentication & Core

- / — GET → Redirect to /login
- /register_patient — GET, POST → Register patient
- /login — GET, POST → Login
- /logout — POST → Logout

2. Admin Routes

- /admin/dashboard — GET, POST → Admin dashboard
- /admin/search — GET → Admin search

- /admin/departments/create — GET, POST → Create department
- /admin/doctors/create — GET, POST → Add doctor
- /admin/doctors/<int:doctor_id>/edit — GET, POST → Edit doctor
- /admin/doctors/<int:doctor_id>/delete — POST → Delete doctor
- /admin/doctors/<int:doctor_id>/force_delete — POST → Force delete doctor
- /admin/doctors/<int:doctor_id>/blacklist — POST → Blacklist doctor
- /admin/doctors/<int:doctor_id>/unblacklist — POST → Un-blacklist doctor
- /admin/patients/<int:patient_id>/edit — GET, POST → Edit patient
- /admin/patients/<int:patient_id>/delete — POST → Delete patient
- /admin/patients/<int:patient_id>/force_delete — POST → Force delete patient
- /admin/patients/<int:patient_id>/blacklist — POST → Blacklist patient
- /admin/patients/<int:patient_id>/unblacklist — POST → Un-blacklist patient
- /admin/appointments — GET → All appointments
- /admin/patients/<int:patient_id>/history — GET → Patient history (completed)
- /admin/appointments/<int:appointment_id> — GET → Detailed appointment view

3. Doctor Routes

- /doctor/<int:doctor_id>/dashboard — GET, POST → Doctor dashboard
- /doctor/<int:doctor_id>/availability — GET, POST → Provide availability
- /appointments/<int:appointment_id>/history/update — GET, POST → Update patient history
- /appointments/<int:appointment_id>/history/view — GET → View patient history
- /doctor/appointments/<int:appointment_id>/cancel — POST → Doctor cancel appointment
- /doctor/<int:doctor_id>/appointments/past — GET → Doctor past appointments

4. Patient Routes

- /patient/<int:patient_id>/dashboard — GET → Patient dashboard
- /patient/<int:patient_id>/search — GET → Search
- /patient/<int:patient_id>/profile/edit — GET, POST → Edit profile
- /patient/<int:patient_id>/departments/<int:dept_id> — GET → Department details
- /patient/doctors/<int:doctor_id> — GET → Doctor details
- /patient/<int:patient_id>/doctors/<int:doctor_id>/availability — GET, POST → Book availability
- /patient/appointments/<int:appointment_id>/cancel — POST → Cancel appointment
- /patient/<int:patient_id>/history — GET → Patient completed history
- /patient/<int:patient_id>/appointments/past — GET → Patient past appointments

6. API Resource Endpoints

Not implemented

How to run the app:

Unzip the project folder, create a virtual environment (optional), install all the required dependencies using **pip install -r requirements.txt**, and then start the application by running **python app.py**.

7. Video Presentation

DRIVE LINK:

<https://drive.google.com/file/d/14eEtIHGWEv0SB1qtoil2QkZ505ZtlblZ/view?usp=sharing>